COS120 - CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BÀI 3: SẮP XẾP

MỤC TIÊU

- Hiểu rõ và thực hiện được các phương pháp sắp xếp.
- Biết vận dụng, lựa chọn phương pháp sắp xếp cho từng bài toán thực tế.
- Cài đặt được các giải thuật sắp xếp trên dữ liệu thực tế.

NỘI DUNG

- 3.1 - GIỚI THIỆU BÀI TOÁN SẮP XẾP
- 3.2 - ĐỔI CHỖ TRỰC TIẾP – INTERCHANGE SORT
- 3.3 - CHỌN TRỰC TIẾP – SELECTION SORT
- 3.4 - CHÈN TRỰC TIẾP – INSERTION SORT
- 3.5 - SẮP XẾP NỔI BỌT – BUBBLE SORT
- 3.6 - SẮP XẾP NHANH – QUICK SORT

SẮP XẾP

3.1 - GIỚI THIỆU BÀI TOÁN SẮP XẾP

GIỚI THIỆU

- Sắp xếp là một công việc quan trọng và phổ biến
- Giúp tìm kiếm, truy cập và xử lý thông tin nhanh chóng, dễ dàng.
- Ví dụ: sắp xếp hàng hóa theo tiêu chí giá cả, tên hàng hóa, ngày nhập kho,…
- Trong CNTT, dữ liệu đã được sắp xếp giúp cho tăng tốc độ tìm kiếm và truy xuất.

MÔ TẢ BÀI TOÁN

- Sắp xếp là sự phân bố lại các phần tử của một tập hợp theo một trình tự nhất định
- Việc sắp xếp dựa theo một tiêu chí, gọi là **khóa** của quá trình sắp xếp.

Ví dụ: sắp xếp **giảm dần** một tập hợp các số nguyên. **Khóa** của sự sắp xếp này chính là giá trị số nguyên của mỗi phần tử.

Tập hợp đầu vào: {10, 4, 8, 4, 9, 3, 12, 2}

10	4	8	4	9	3	12	2
0	1	2	3	4	5	6	7

Kết quả sau khi **sắp xếp giảm dần**: {12, 10, 9, 8, 4, 4, 3, 2}

12	10	9	8	4	4	3	2
0	1	2	3	4	5	6	7

MÔ TẢ BÀI TOÁN

- Ví dụ: Thông tin một hàng hóa như sau: (MaHangHoa, TenHangHoa, DonGia).
- Ta có thể sắp xếp một tập hợp hàng hóa theo tiêu chí:
  - Mã hàng tăng dần
  - Hoặc tên hàng theo thứ tự alphabet
  - Hoặc sắp xếp giảm dần theo đơn giá.

- Kết quả sắp xếp theo **mã hàng hóa**:

HH01 Bàn phím 67 USD	HH03 Màn hình 209 USD	HH09 Chuột 35 USD	HH10 RAM 114 USD	HH11 Loa 38 USD	HH15 Đĩa SSD 59 USD	HH20 Bộ sạc 51 USD	HH21 Thẻ nhớ 8 USD
0	1	2	3	4	5	6	7

KHÁI NIỆM NGHỊCH THẾ

- Xét trong **bài toán sắp xếp tăng dần**, một cặp số nguyên trong mảng a được gọi là một **nghịch thế** nếu: **i < j** **nhưng** **a[i] > a[j].**

12	4	1	5	3	8	2	2
0	1	2	3	4	5	6	7

- (a[0]=12, a[1]=4) là một nghịch thế. Vì 0<1 nhưng a[0]>a[1].
- (a[2], a[3]) không là nghịch thế.
- (a[3], a[4]) là một nghịch thế.
- (a[6], a[7]) không là nghịch thế.
- (a[3], a[6]) là một nghịch thế.

CÁC GIẢI THUẬT SẮP XẾP

- Có nhiều giải thuật sắp xếp, mỗi giải thuật có ưu điểm và hạn chế riêng.
- Một số giải thuật sắp xếp phổ biến và nổi tiếng như: Interchage Sort (đổi chỗ trực tiếp), Selection Sort (chọn trực tiếp), Insertion Sort (chèn trực tiếp), Bubble Sort (nổi bọt), Quick Sort (sắp xếp nhanh có thực hiện phân hoạch), Heap Sort (sắp xếp cây), Shell Sort (sắp xếp với độ dài bước giảm dần), Merge Sort (trộn trực tiếp), Radix Sort (sắp xếp theo phương pháp cơ số),…
- Hai tiêu chí quan trọng đánh giá một giải thuật sắp xếp:
  - **Độ phức tạp thời gian (Time Complexity)**:
  - **Độ phức tạp không gian (Space Complexity)**:

SẮP XẾP

3.2 - ĐỔI CHỖ TRỰC TIẾP (INTERCHANGE SORT)

GIẢI THUẬT

Ý TƯỞNG

- Xuất phát từ **đầu dãy**.
- Tìm tất cả **nghịch thế** chứa phần tử.
- Triệt tiêu chúng bằng cách **hoán vị hai phần tử trong cặp nghịch thế**
- Lặp lại các bước trên cho đến khi không còn nghịch thế.
- Quá trình này hoàn thành thì dãy số đã được sắp xếp tăng.

GIẢI THUẬT

CÁC BƯỚC THỰC HIỆN

```
Bước 1 : i = 0;// bắt đầu từ đầu dãy
Bước 2 : j = i+1;
Bước 3 : //tìm các nghịch thế a[i] > a[j], i<j
     Trong khi j < n thực hiện
          Nếu a[i]>a[j]: Hoán vị a[i], a[j];
          j = j+1;
Bước 4 : i = i+1;
     Nếu  i < n-1: Lặp lại Bước 2.
     Ngược lại:  Dừng.
```

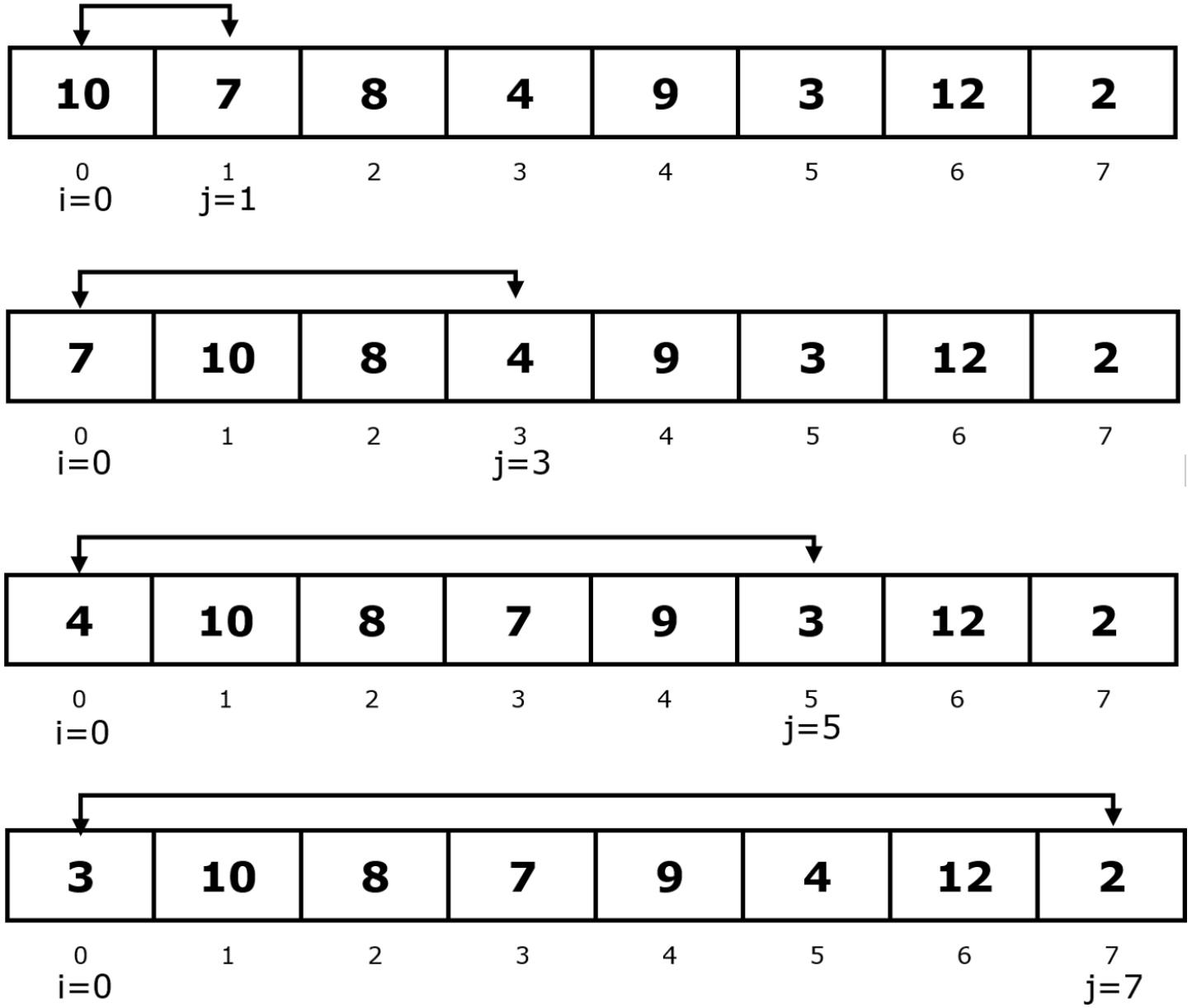
MINH HỌA

- Ví dụ, thực hiện Interchange Sort

Mảng a = {10, 7, 8, 4, 9, 3, 12, 2}

Lần lặp i=0:

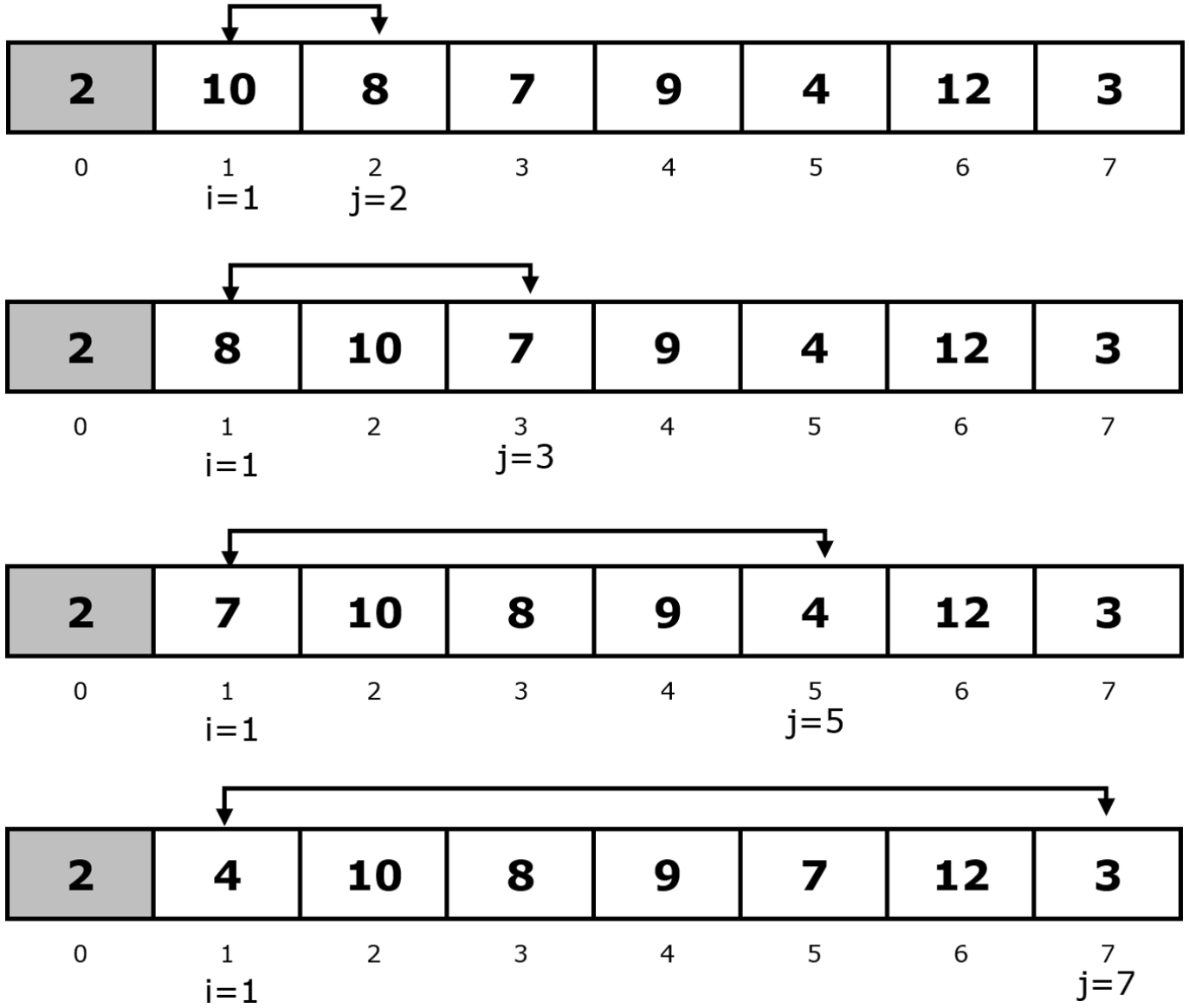
- Cho i đứng yên tại i=0.
- Cho j bắt từ vị trí i+1
- Rồi j lần lượt chạy đến hết mảng để tìm nghịch thế (a[i],a[j]) mà hoán vị.



MINH HỌA

Lần lặp i=1:

- Phần tử ở a[0] đã ổn định.
- Cho i đứng yên tại i=1.
- Cho j bắt từ vị trí i+1 rồi j lần lượt chạy đến hết mảng để tìm nghịch thế (a[i],a[j]) mà hoán vị.



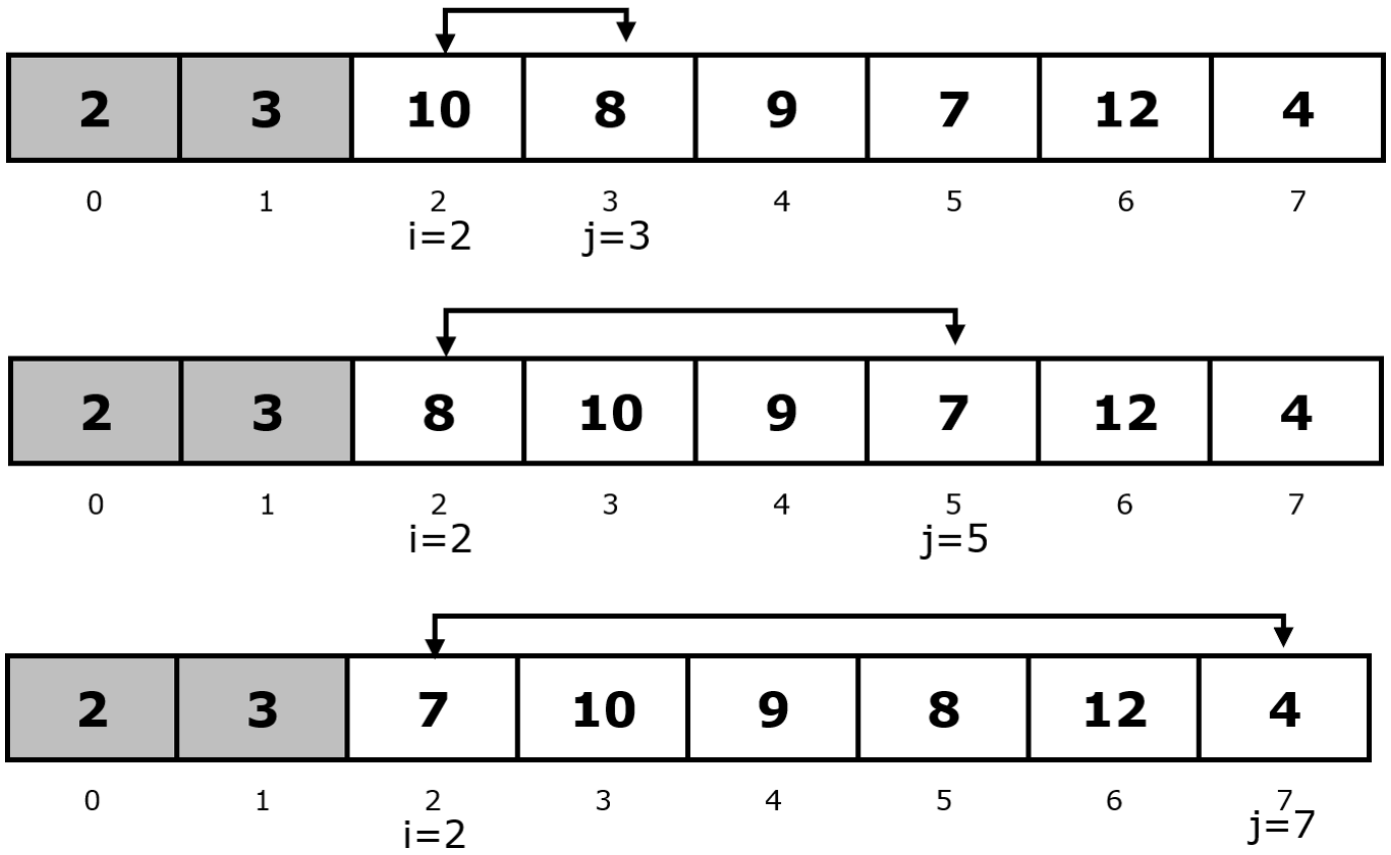
MINH HỌA

Lần lặp i=2:

-Phần tử ở a[0], a[1] đã ổn định.

-Cho i đứng yên tại i=2.

-Cho j bắt từ vị trí j=3.



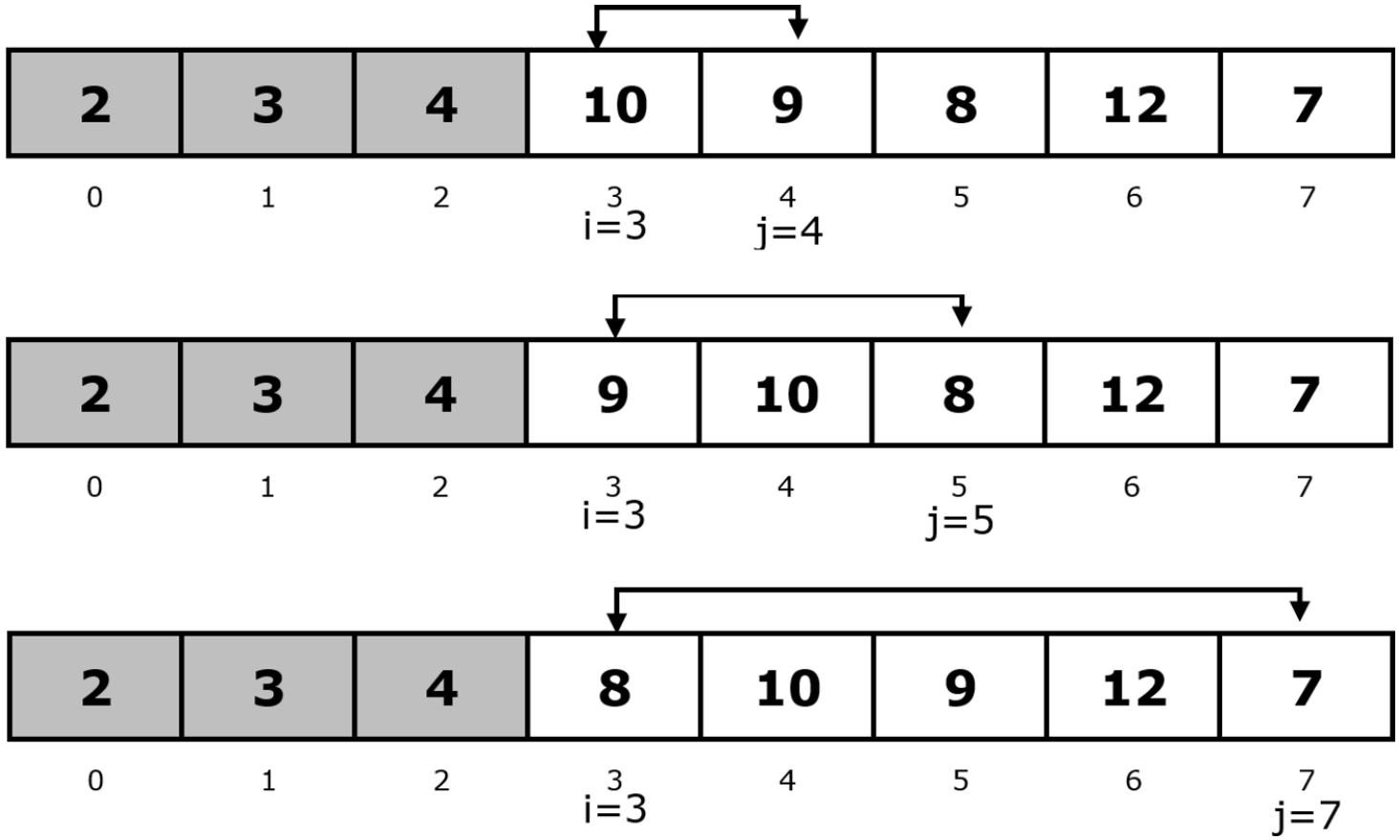
MINH HỌA

Lần lặp i=3:

-Phần tử ở a[0], a[1], a[2] đã ổn định.

-Cho i đứng yên tại i=3.

-Cho j bắt từ vị trí j=4.



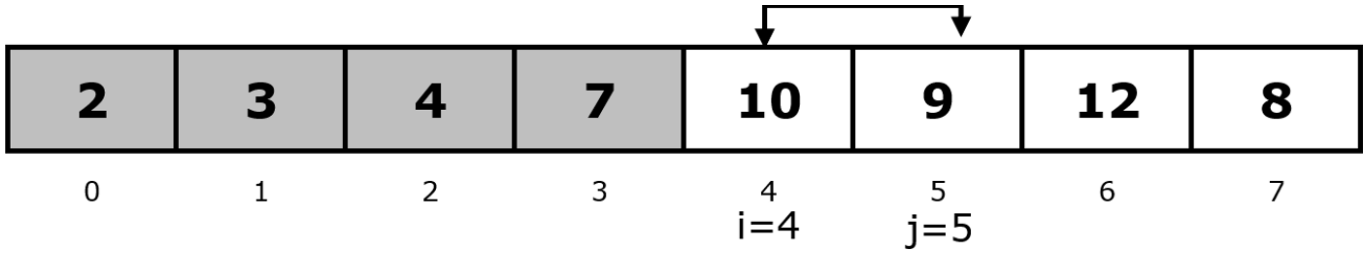
MINH HỌA

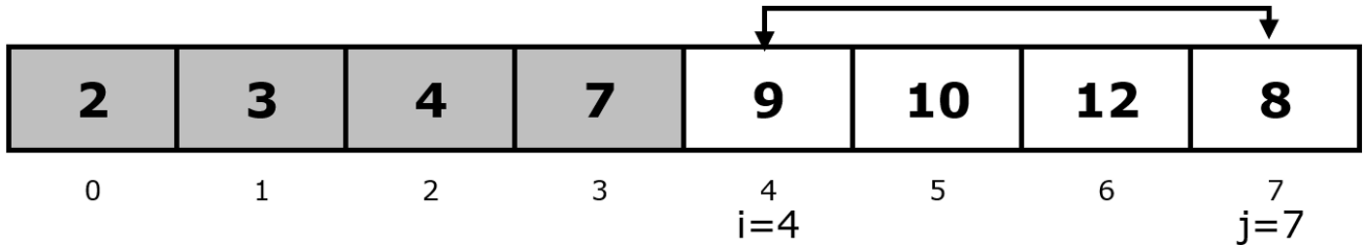
Lần lặp i=4:

-a[0], a[1], a[2], a[3] ổn định.

-Cho i đứng yên tại i=4.

-Cho j bắt từ vị trí j=5.



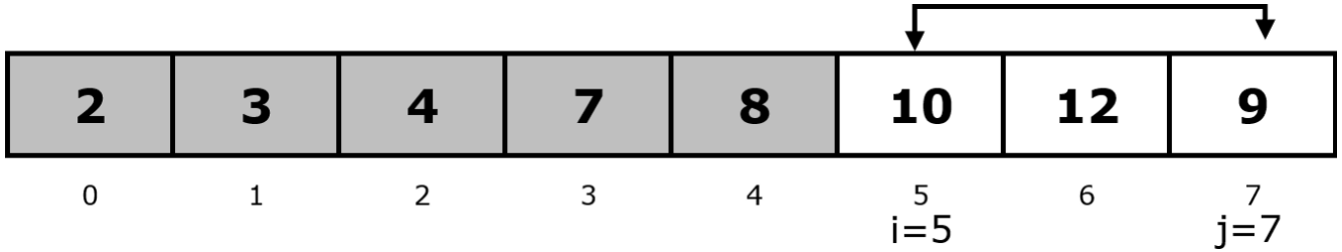


Lần lặp i=5:

-a[0], a[1], a[2], a[3], a[4] ổn định.

-Cho i đứng yên tại i=5.

-Cho j bắt từ vị trí j=6.



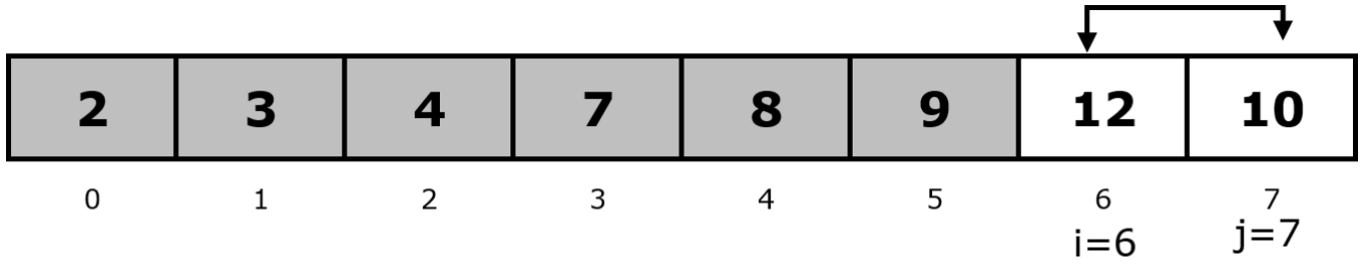
MINH HỌA

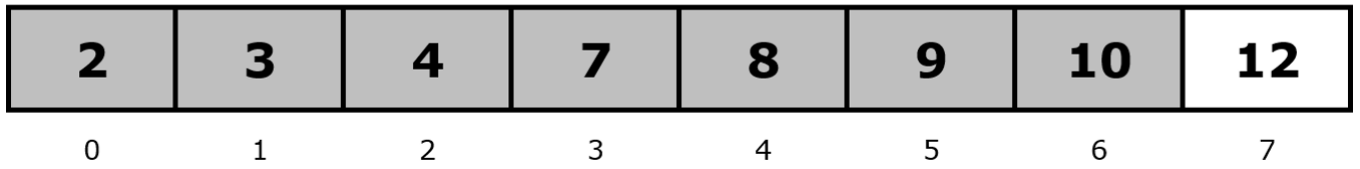
Lần lặp i=6:

-Phần tử ở a[0], a[1], a[2], a[3], a[4], a[5] đã ổn định.

-Cho i đứng yên tại i=6.

-Cho j bắt từ vị trí j=7.





Đến bước này thì mảng đã được sắp xếp tăng, cho nên i không cần chạy đến phần tử cuối mảng nữa.

CÀI ĐẶT

```
void InterchangeSort(int a[], int n)
{
        int i, j;
        for (i = 0 ; i<n-1 ; i++)
             for (j =i+1; j<n ; j++)
                  if(a[i] > a[j])
                       Hoanvi(a[i],a[j]);
}

void Hoanvi(int &x, int &y)
{     int tam=x;
      x=y;
      y=tam;
}
```

NHẬN XÉT

TRƯỜNG HỢP	SỐ LẦN SO SÁNH	SỐ LẦN HOÁN VỊ
Tốt nhất	$\sum_{i=1}^{n-1}(n-i+1) = \frac{n(n-1)}{2}$	0
Xấu nhất	$\frac{n(n-1)}{2}$	$\sum_{i=1}^{n-1}(n-i+1) = \frac{n(n-1)}{2}$

- Độ phức tạp của Interchange Sort được biểu diễn bằng $O(n^2)$

BÀI TẬP tại cá nhân tại lớp

- Áp dụng thuật toán **Interchange Sort** cho mảng a và ghi ra kết quả từng bước

Mảng a = **{10, 5, 30, 70, 40, 80, 90}**

SẮP XẾP

3.3 - CHỌN TRỰC TIẾP (SELECTION SORT)

GIẢI THUẬT

Ý TƯỞNG

Nếu mảng đã được **sắp tăng** thì phần tử **a[0]** là giá trị **min** của đoạn (**a[0] đến a[n-1]).**

- Đầu tiên, tìm **min** của đoạn (**a[0]** đến a[n-1]) đưa nó về **đầu đoạn**.
- Bước tiếp theo, tìm min của đoạn (**a[1]** đến a[n-1]), đưa nó về **đầu đoạn**.
- Lần lượt lặp lại quá trình trên cho đoạn còn lại...
- Đến khi đoạn cuối cùng chỉ còn lại 1 phần tử thì dừng.

GIẢI THUẬT

CÁC BƯỚC THỰC HIỆN

```
Bước 1:
    i = 0;
Bước 2:
    Tìm phần tử nhỏ nhất là a[min] trong đoạn từ  a[i] đến a[n-1]
Bước 3:
    Hoán vị a[min] và a[i]
Bước 4:
    i = i+1
    Nếu  i<n-1 thì lặp lại Bước 2
    Ngược lại: Dừng. //n phần tử đã nằm đúng vị trí.
```

MINH HỌA

Cho mảng số nguyên

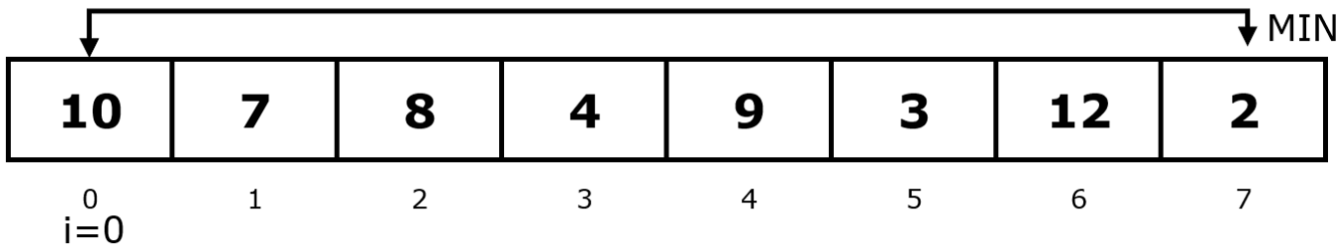
a = {10, 7, 8, 4, 9, 3, 12, 2}.

Thực hiện Selection Sort cho nó.

Lần lặp i=0:

Cho i đứng yên tại i=0. Tìm phần tử nhỏ nhất a[min] trong đoạn (a[0] đến a[7]).

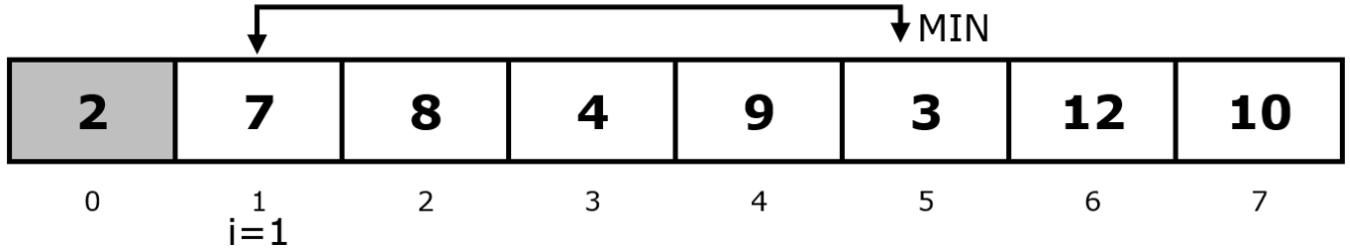
Thực hiện hoán vị a[i] và a[min].



Lần lặp i=1:

Cho i đứng yên tại i=1. Tìm phần tử nhỏ nhất a[min] trong đoạn (a[1] đến a[7]).

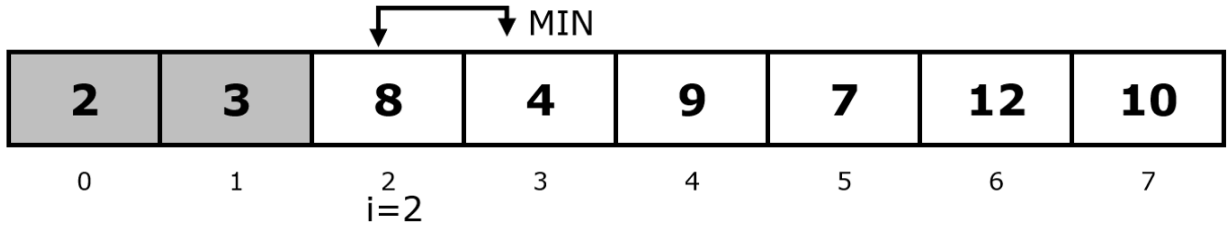
Thực hiện hoán vị a[i] và a[min].



MINH HỌA

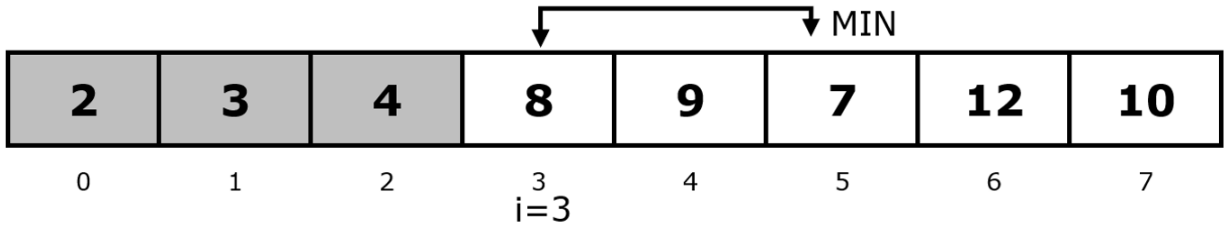
Lần lặp i=2:

Cho i đứng yên tại i=2. Tìm phần tử nhỏ nhất a[min] trong đoạn (a[2] đến a[7]).
Thực hiện hoán vị a[i] và a[min].

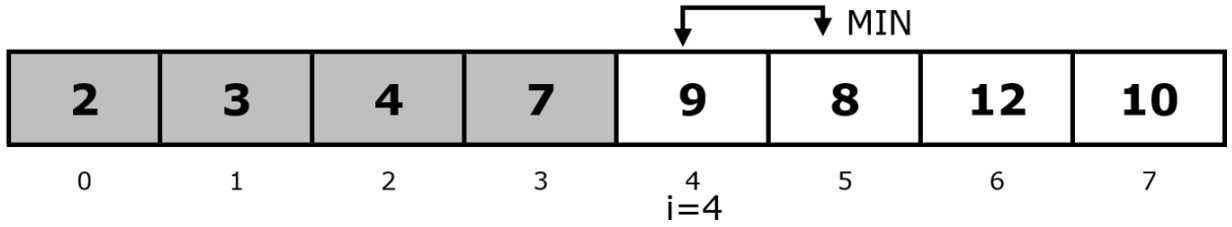


Lần lặp i=3:

Thực hiện tương tự như các lần lặp trước.

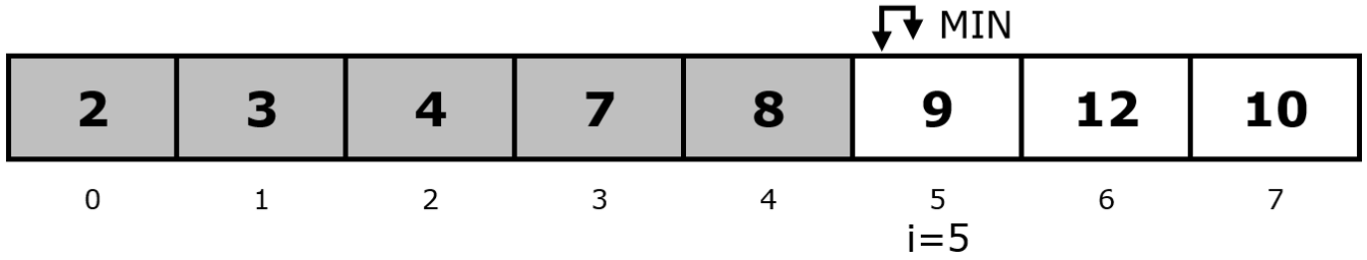


Lần lặp i=4:

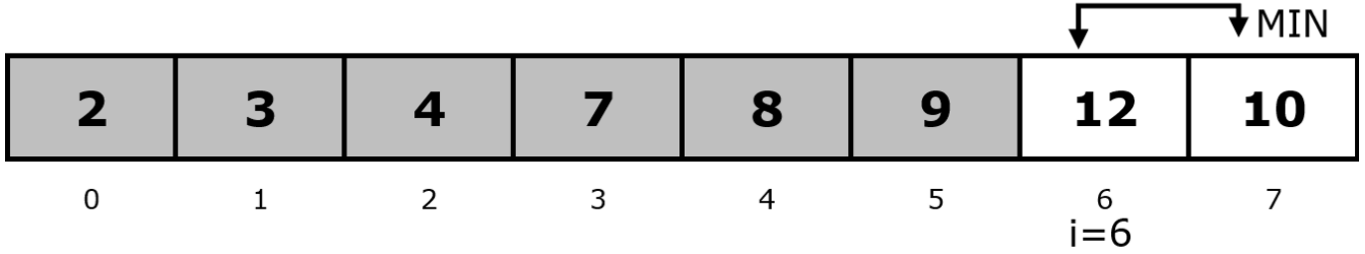


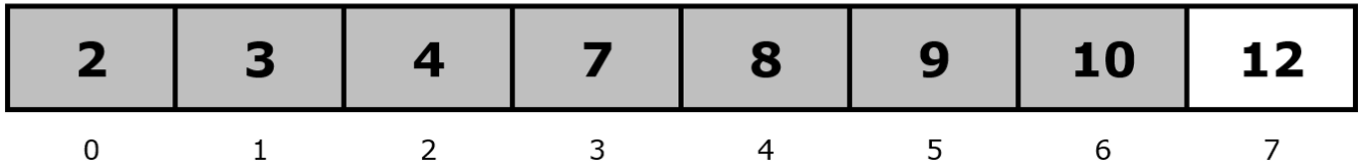
MINH HỌA

Lần lặp i=5:



Lần lặp i=6:





Ở lần lặp thứ i=6 này, sau khi hoán vị a[i] và phần tử a[min] thì mảng đã sắp xếp xong. Do đó không cần phải di chuyển tiếp i đến phần tử cuối của mảng.

CÀI ĐẶT

```
void SelectionSort(int a[],int n)
{
    int min;
    for (int  i=0; i<n-1 ; i++)
    {
        min = i;
        for(int j = i+1; j <n ; j++)
            if (a[j] < a[min])
                min=j;
        Hoanvi(a[min], a[i]);
    }
}
```

NHẬN XÉT

- Trong mọi trường hợp ta có thể tính số lượng phép so sánh như sau

$$\sum_{i=1}^{n-1}(n-i+1) = \frac{n(n-1)}{2}$$

TRƯỜNG HỢP	SỐ LẦN SO SÁNH	SỐ LẦN GÁN
Tốt nhất	n(n-1)/2	0
Xấu nhất	n(n-1)/2	3n(n-1)/2

- Độ phức tạp của Selection Sort được biểu diễn bằng $O(n^2)$

BÀI TẬP tại cá nhân tại lớp

- Áp dụng thuật toán **SELECTION SORT** cho mảng a và ghi ra kết quả từng bước

Mảng a = **{10, 5, 30, 70, 40, 80, 90}**

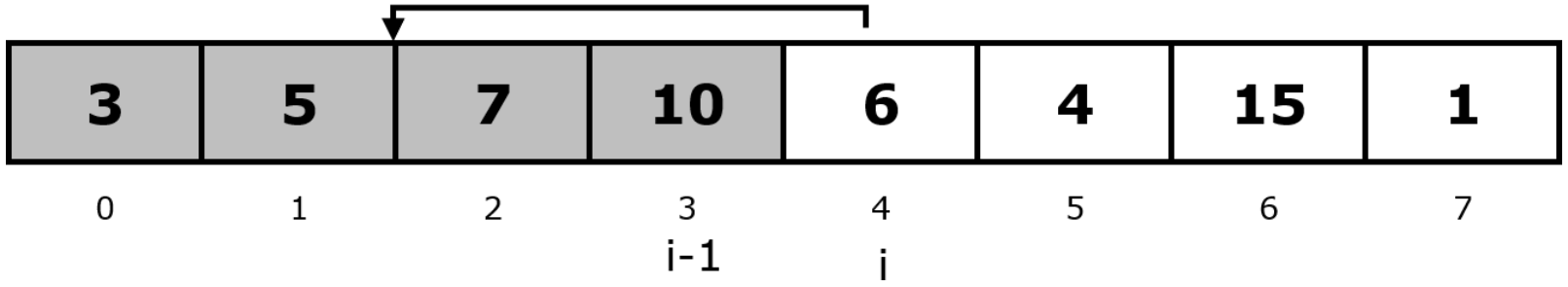
SẮP XẾP

3.4 - CHÈN TRỰC TIẾP (INSERTION SORT)

GIẢI THUẬT

Ý TƯỞNG

- Cho trước một mảng **chưa** sắp xếp hoàn chỉnh.
- Giả sử đoạn **(a[0], a[i-1])** đã **được** sắp xếp.
- Ta gọi đoạn này là **mảng con**.



- Lấy phần tử a[i]
- Tìm **vị trí** thích hợp để **chèn a[i]** vào **mảng con** sao cho sau khi chèn thì **mảng con cũng tăng**.
- Lại tiếp tục với **a[i+1]** để thực hiện chèn tương tự như vậy.
- Rồi lần lượt thực hiện tương tự với các phần tử còn lại của mảng lớn cho đến hết mảng.

GIẢI THUẬT

CÁC BƯỚC THỰC HIỆN

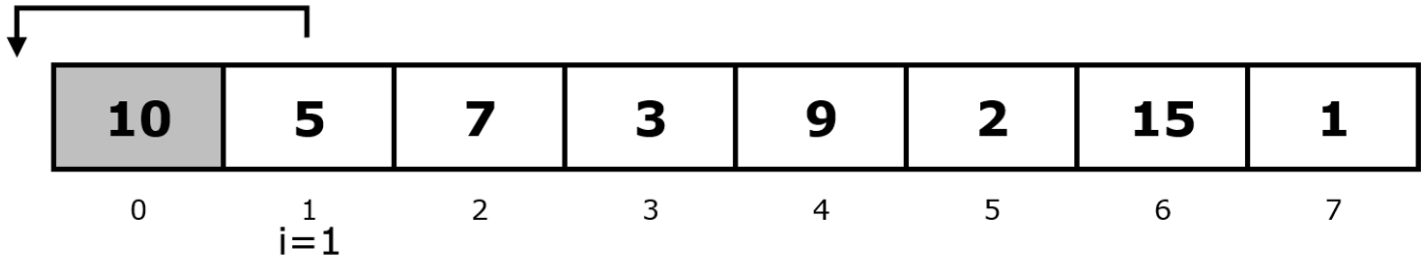
- Có thể xem đoạn từ **a[0] đến a[0]** là mảng con đã sắp tăng.
- Ta bắt đầu chọn phần tử tại **vị trí 1** để chèn vào mảng con này.

```
Bước 1:  i = 1;     // giả sử có đoạn a[0] đã được sắp tăng
Bước 2: x = a[i]; //cất a[i] vào x
        Tìm vị trí pos thích hợp trong đoạn
          từ a[0] đến a[i-1] để chèn a[i] vào
Bước 3: Dời chỗ các phần tử  từ a[pos] đến a[i-1]
          sang phải một vị trí để dành chỗ cho a[i]
Bước 4: a[pos] = x; //có đoạn a[0] đến a[i] đã được sắp tăng
Bước 5:  i=i+1;
         Nếu  i<n : Lặp lại Bước 2.
         Ngược lại : Dừng.
```

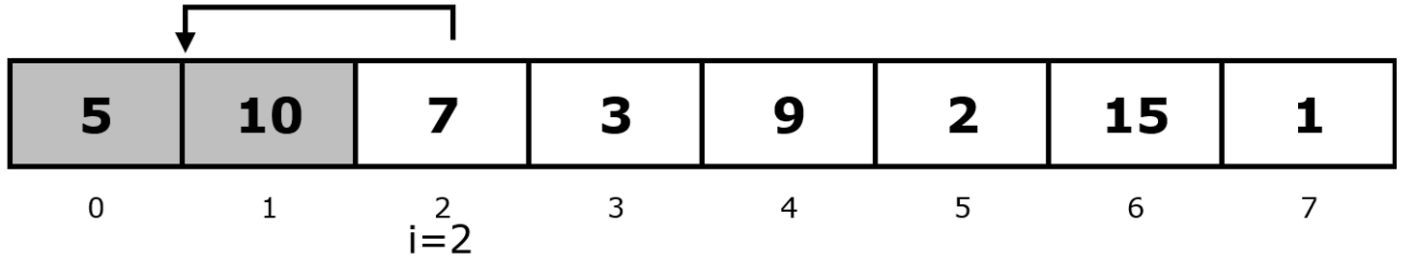
MINH HỌA

- Mảng a = {10, 5, 7, 3, 9, 2, 15, 1}.
- Mảng 1 phần tử a[0] đã sắp xếp tăng.
- Giải thuật sẽ bắt đầu chạy từ i=1.

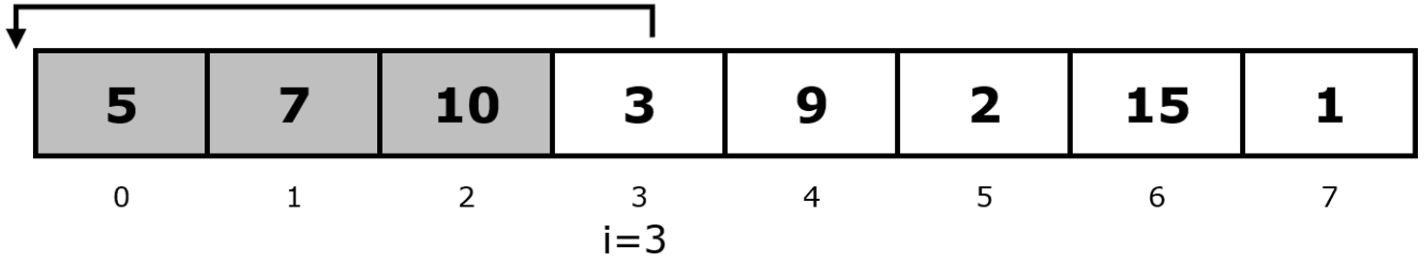
Lần lặp i=1:



Lần lặp i=2:

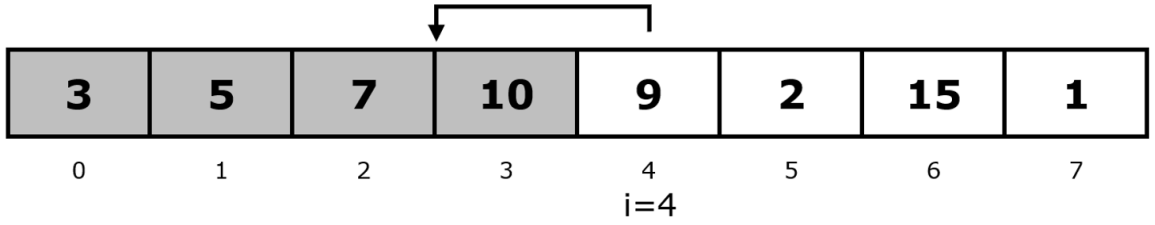


Lần lặp i=3:

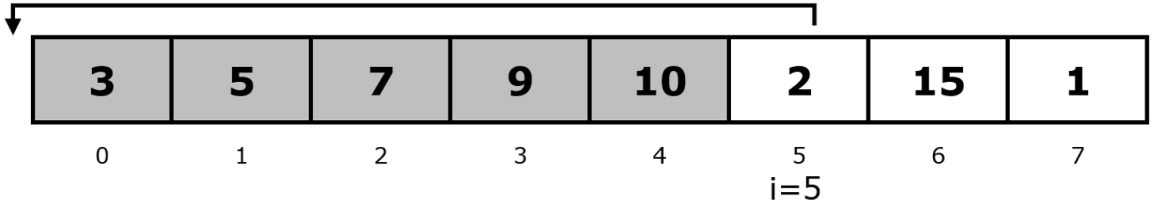


MINH HỌA

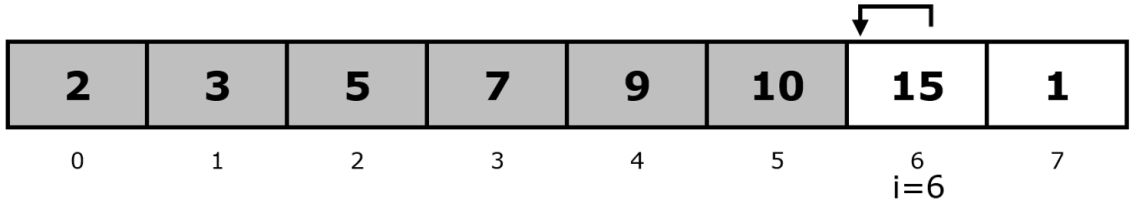
Lần lặp i=4:



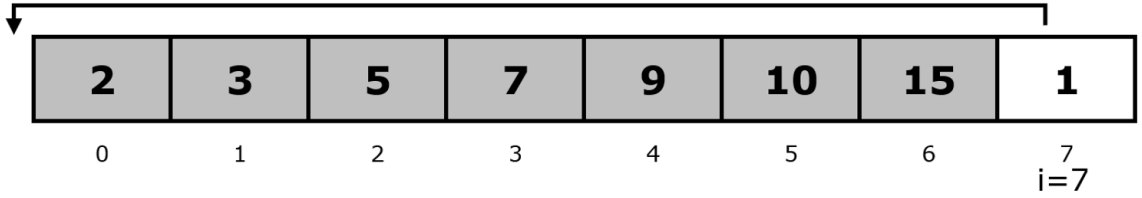
Lần lặp i=5:

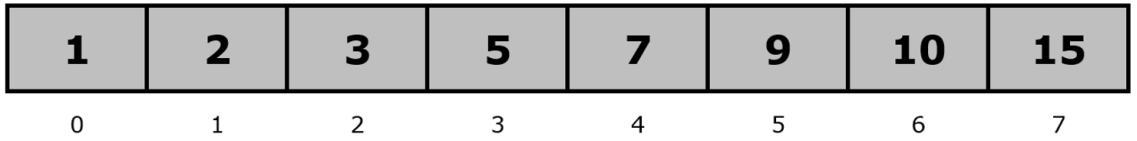


Lần lặp i=6:



Lần lặp i=7:





CÀI ĐẶT

```
1   void InsertionSort(int a[], int n)
2   {
3       int pos;
4       int x; //lưu a[i] vào x để tránh bị ghi đè khi dời chỗ
5       for(int i=1 ; i<n ; i++) //đoạn a[0] đã sắp tăng
6       {
7           x = a[i];
8           pos = i-1;
9           while((pos>=0) && (a[pos]>x))// tìm vị trí chèn x
10          {
11          //kết hợp dời chỗ các phần tử sẽ đứng sau x trong dãy mới
12              a[pos+1] = a[pos];
13              pos--;
14          }
15          a[pos+1] = x;   //chèn x vào dãy
16      }
17  }
```

NHẬN XÉT

TRƯỜNG HỢP	SỐ LẦN SO SÁNH	SỐ LẦN GÁN
Tốt nhất	$\sum_{i=1}^{n-1} 1 = n - 1$	$\sum_{i=1}^{n-1} 2 = 2(n-1)$
Xấu nhất	$\sum_{i=1}^{n-1} (i-1) = \frac{n(n-1)}{2}$	$\sum_{i=1}^{n-1} (i+1) = \frac{n(n+1)}{2} - 1$

- Độ phức tạp của Insertion Sort được biểu diễn bằng $O(n^2)$

BÀI TẬP tại cá nhân tại lớp

- Áp dụng thuật toán **INSERTION SORT** cho mảng a và ghi ra kết quả từng bước

Mảng a = **{10, 5, 30, 70, 40, 80, 90}**

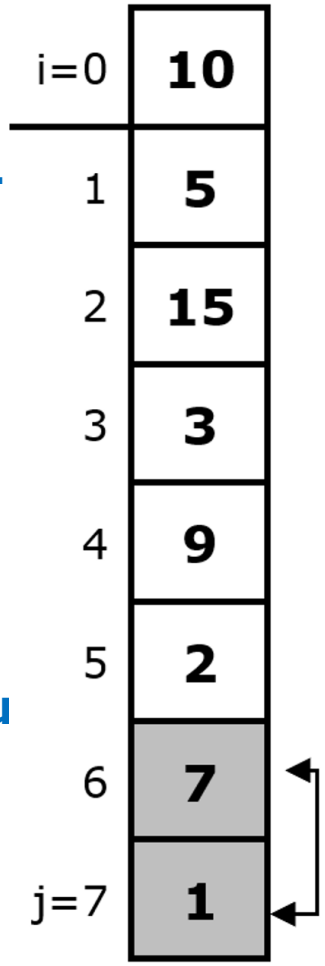
SẮP XẾP

3.5 - SẮP XẾP NỔI BỌT (BUBBLE SORT)

GIẢI THUẬT

Ý TƯỞNG

- Bong bóng nước từ **đáy hồ** sẽ nổi dần lên **mặt hồ**, còn cái gì **nặng** thì **chìm xuống**.
- Người ta dùng ý tưởng đó thiết kế ra phương pháp **sắp xếp nổi bọt** (Bubble Sort).
- Ta dựng đứng mảng lên để dễ hình dung, **a[0]** ở mặt hồ, **a[n-1]** ở đáy hồ.
- **Bắt đầu từ đáy**, ta xét hai phần tử liền nhau, nếu chúng **nghịch thế** thì ta **hoán vị** chúng.
- Ta di chuyển lên thêm một bước, tiến hành làm tương tự cho **hai phần tử liền nhau**
- Cứ như vậy, sẽ có phần tử **min nổi tại vị trí 0**. Như vậy a[0] đã ổn định.
- Ta tiếp tục thực hiện sự nổi bọt **từ đáy** cho dãy (a[1], a[n-1])
- Cứ tiếp tục như vậy cho đến khi mảng được sắp tăng hoàn toàn.



GIẢI THUẬT

CÁC BƯỚC THỰC HIỆN

```
Bước 1:    i = 0;
Bước 2:    j = n-1;
           Trong khi (j>i) thực hiện:
                Nếu a[j]<a[j-1]: Hoán vị a[j] và a[j-1]
                j = j-1;
Bước 3:     i = i+1;
           Nếu  i >= n-1: Dừng
           Ngược lại: Lặp lại Bước 2.
```

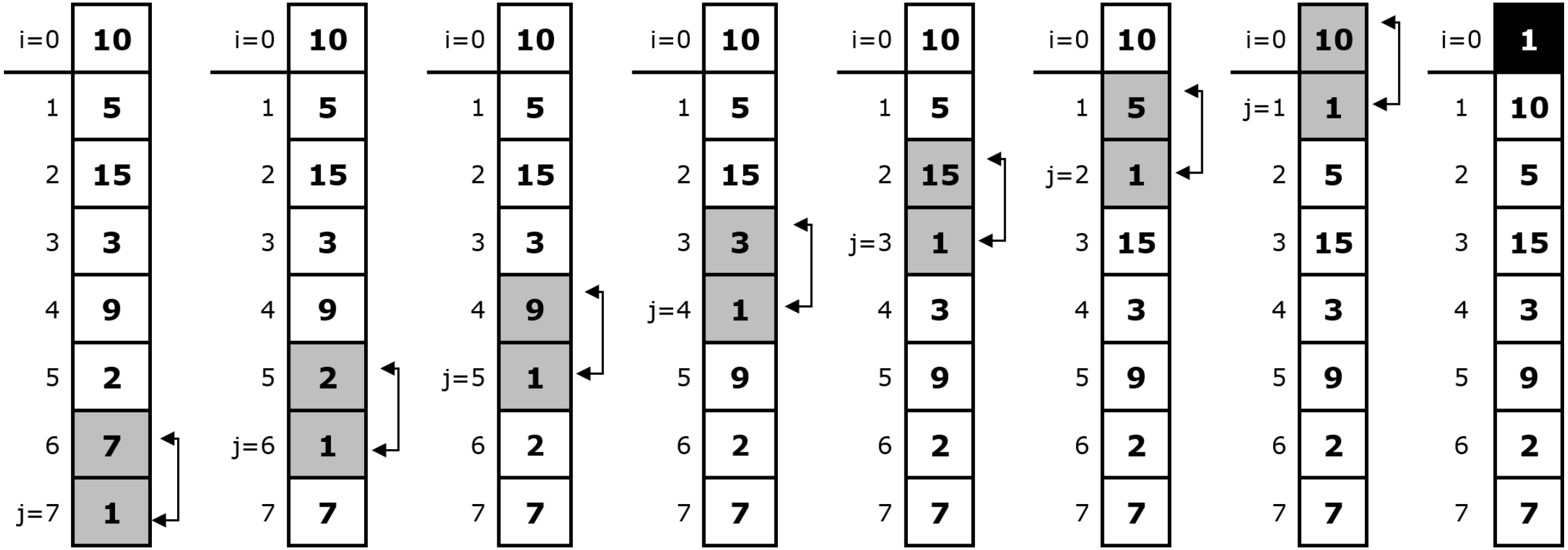
MINH HỌA

- Ví dụ: cho mảng số nguyên a = {10, 5, 15, 3, 9, 2, 7, 1}.

Ta dựng đứng mảng này lên.

Lần lặp i=0:

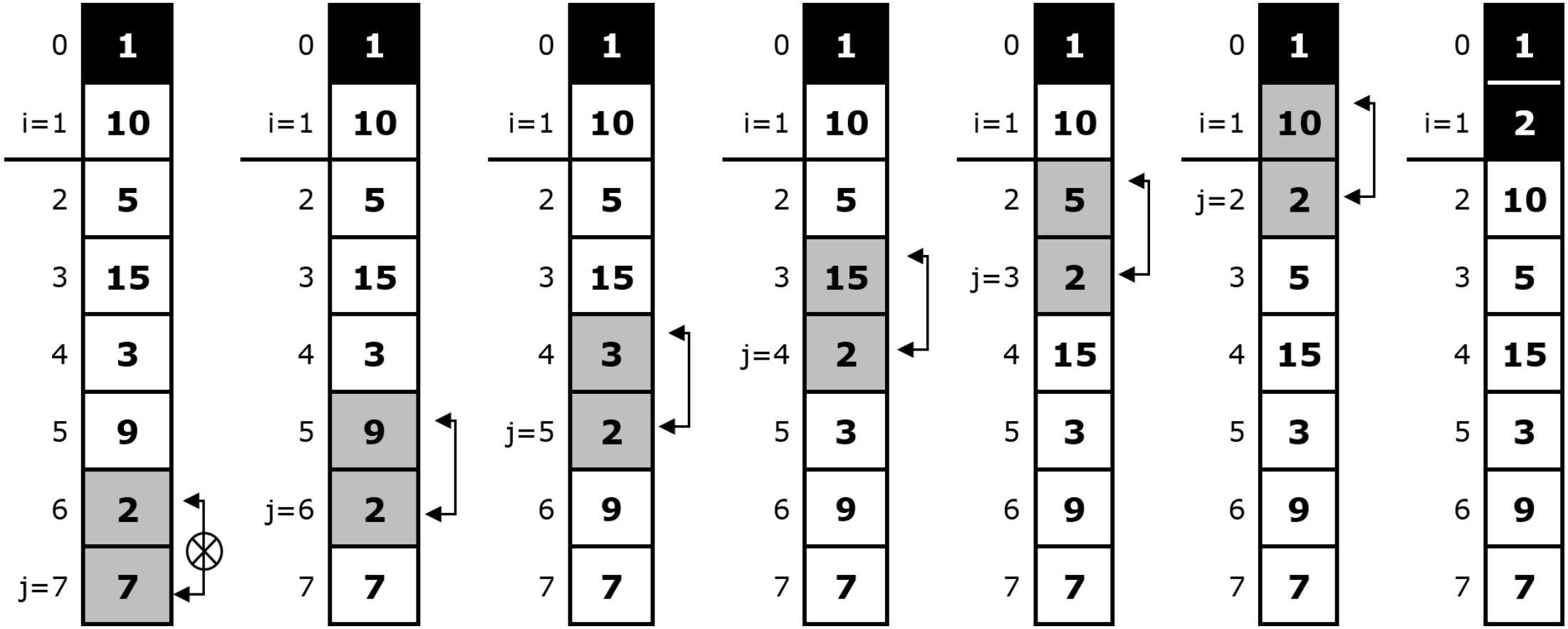
Cho i đứng yên tại i=0, j bắt đầu chạy từ vị trí đáy là j=7 chạy ngược dần lên



MINH HỌA

Lần lặp i=1:

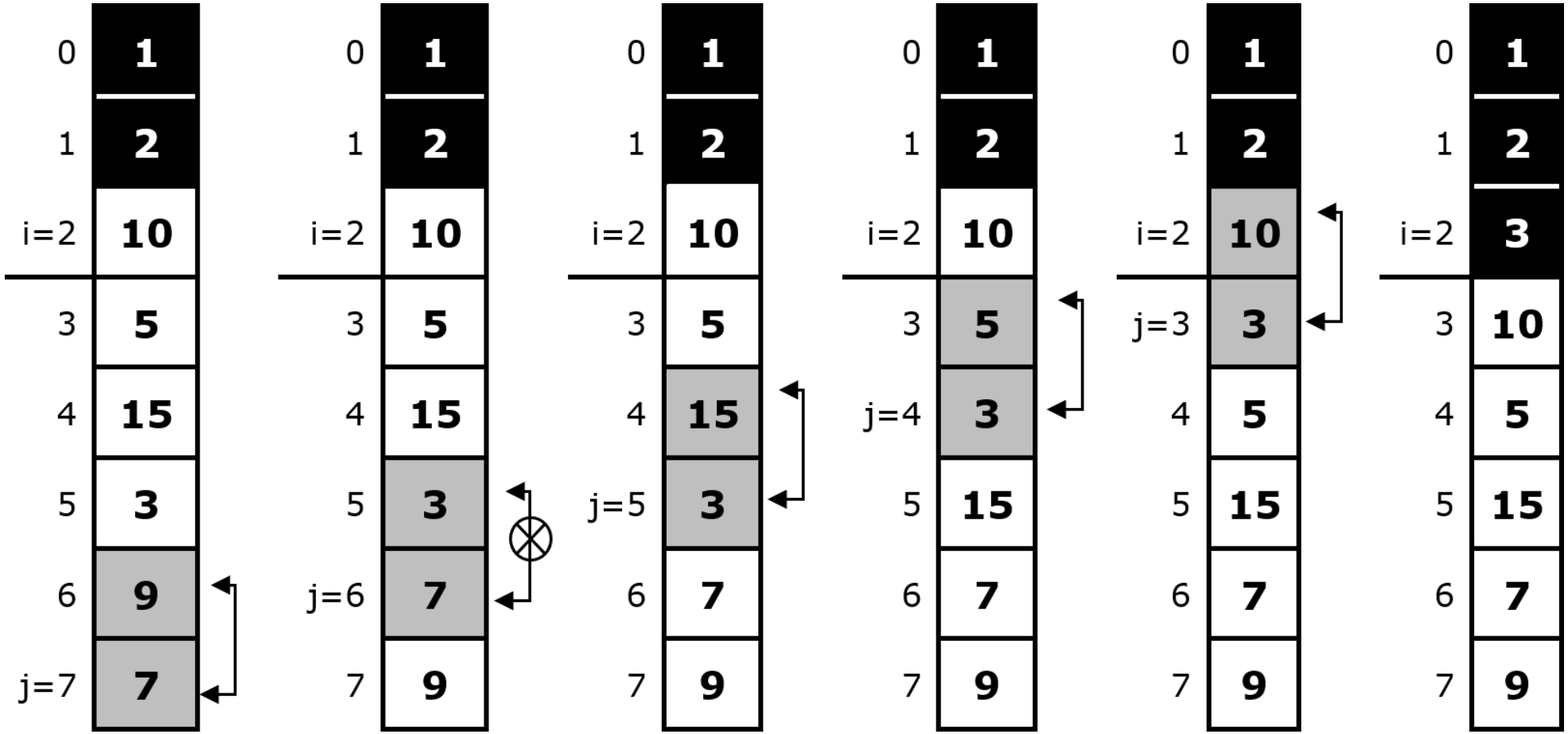
- Phần tử a[0] đã đúng vị trí, ta không quan tâm nó nữa.
- Cho i đứng yên tại i=1, j bắt đầu chạy từ j=7 chạy ngược dần lên.



MINH HỌA

Lần lặp i=2:

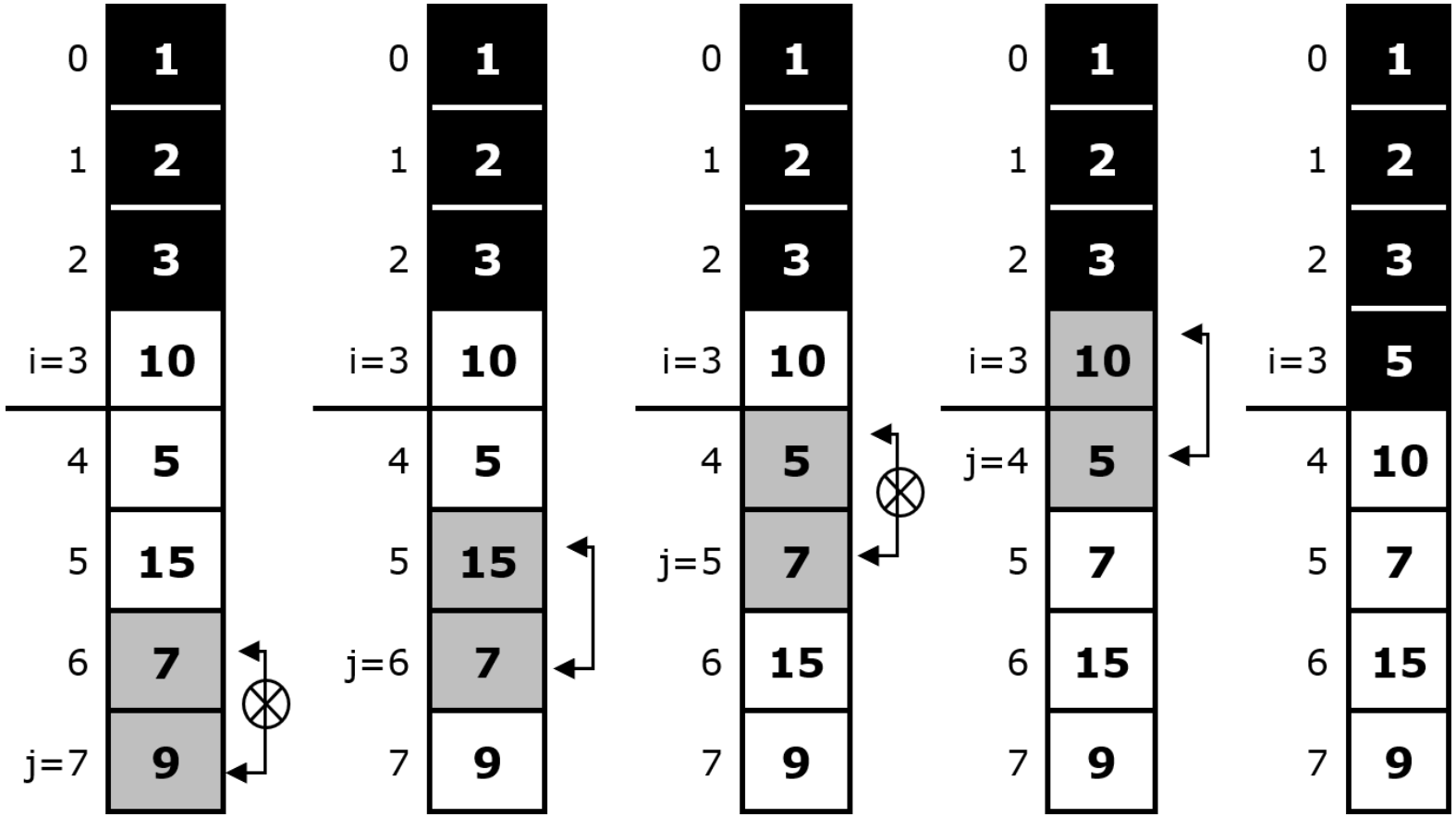
- Các phần tử a[0], a[1] đã đúng vị trí, ta không quan tâm chúng nữa.
- Cho i đứng yên tại i=2, j bắt đầu chạy từ j=7 chạy ngược dần lên.



MINH HỌA

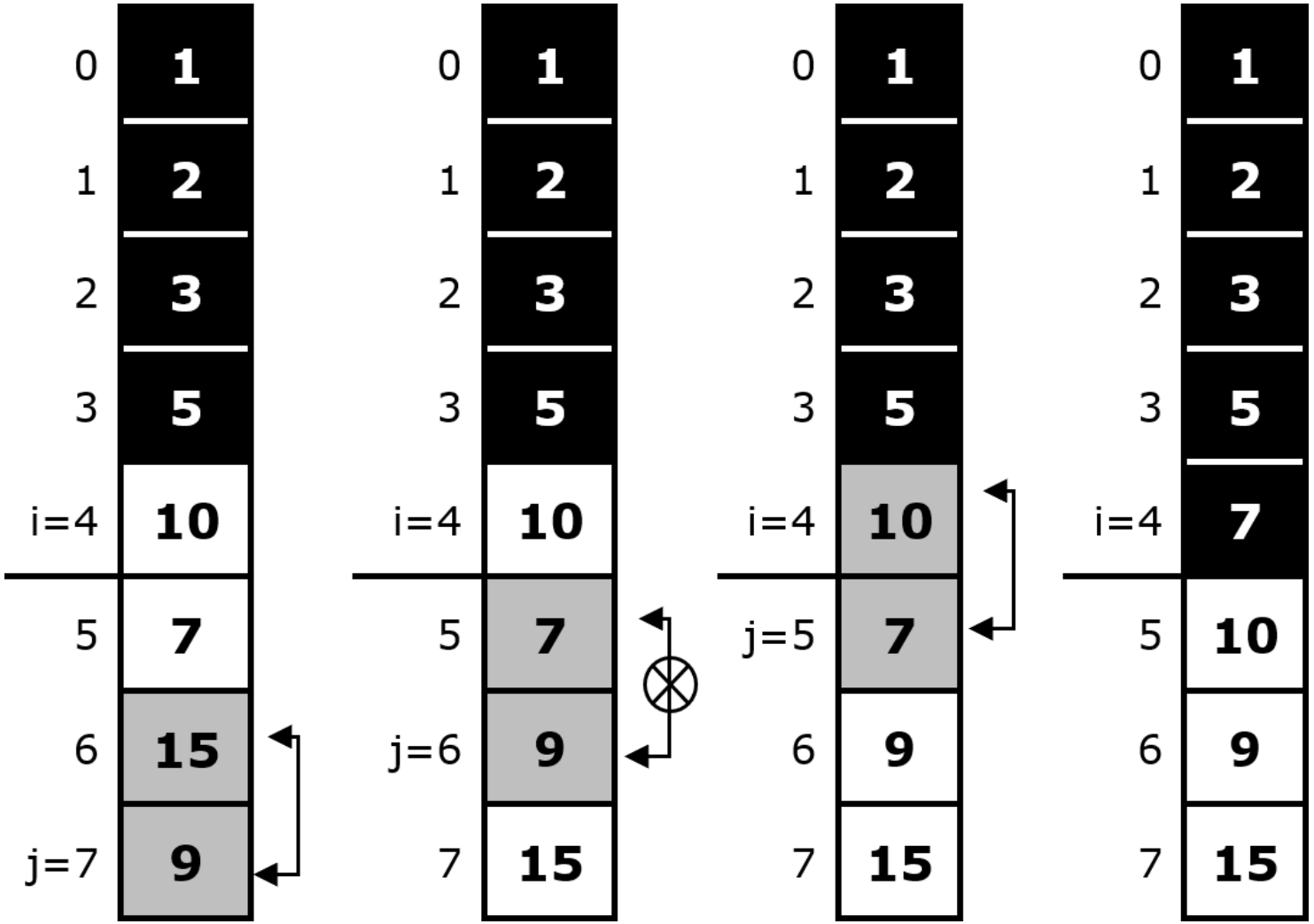
Lần lặp i=3:

- Các phần tử a[0], a[1], a[2] đã đúng vị trí, ta không quan tâm chúng nữa.
- Cho i đứng yên tại i=3, j bắt đầu chạy từ j=7 chạy ngược dần lên.



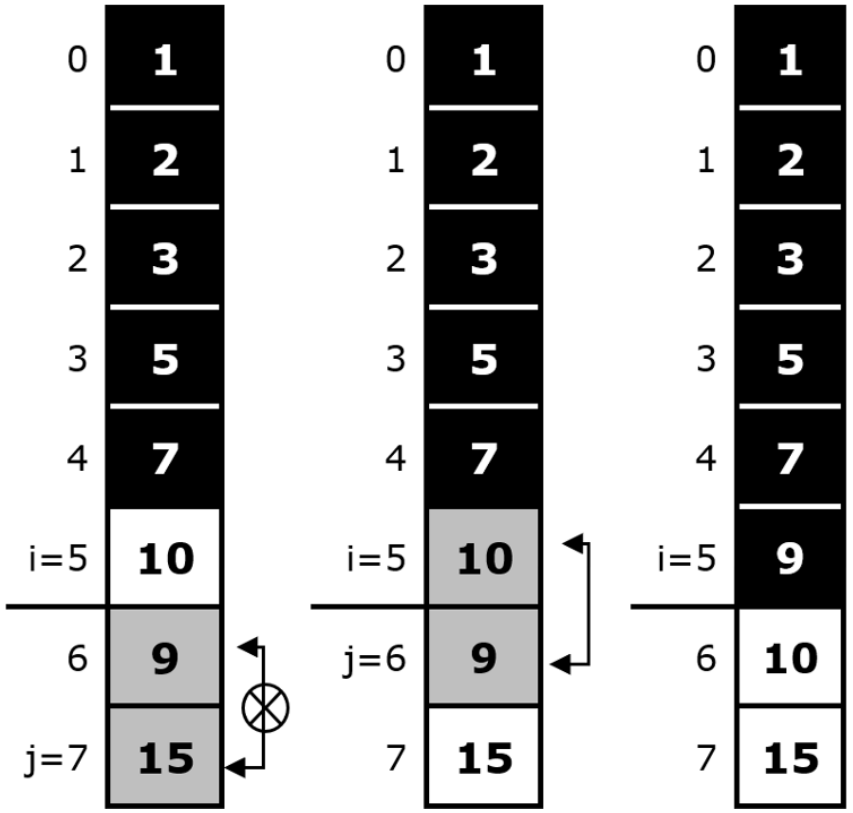
MINH HỌA

- **Lần lặp i=4:**

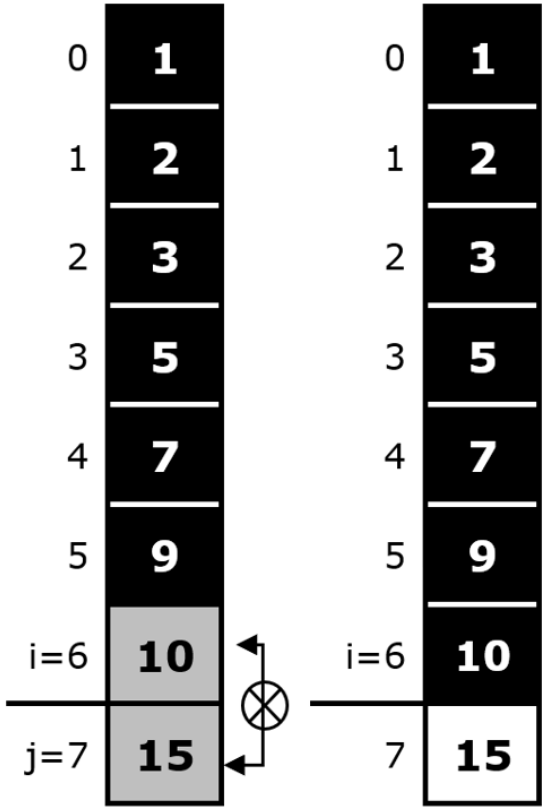


MINH HỌA

Lần lặp i=5:



Lần lặp i=6:



- Kết thúc lần lặp i=6 thì phần tử cuối cũng đã là phần tử lớn nhất rồi.
- Như vậy mảng đã được sắp xếp tăng. Giải thuật dừng.

CÀI ĐẶT

```
1  void BubleSort(int a[], int n)
2  {
3          int i, j;
4          for (i=0 ; i<n-1 ; i++)
               for (j=n-1; j>i ; j--)
5                  if(a[j]< a[j-1])
6                      Hoanvi(a[j],a[j-1]);
7  }
```

NHẬN XÉT

TRƯỜNG HỢP	SỐ LẦN SO SÁNH	SỐ LẦN HOÁN VỊ
Tốt nhất	$\sum_{i=1}^{n-1}(n-i+1)=\frac{n(n-1)}{2}$	0
Xấu nhất	$\frac{n(n-1)}{2}$	$\sum_{i=1}^{n-1}(n-i+1)=\frac{n(n-1)}{2}$

- Độ phức tạp của Bubble Sort được biểu diễn bằng $O(n^2)$

BÀI TẬP tại cá nhân tại lớp

- Áp dụng thuật toán **BUBBLE SORT** cho mảng a và ghi ra kết quả từng bước

Mảng a = **{10, 5, 30, 70, 40, 80, 90}**

SẮP XẾP

3.6 - SẮP XẾP NHANH (QUICK SORT)

GIẢI THUẬT

Ý TƯỞNG

- Quick Sort là giải thuật rất hiệu quả, thông dụng. Thời gian chạy của giải thuật tốt hơn các giải thuật đã đề cập phía trên.
- Phương pháp của giải thuật này như sau:

Mảng con thứ nhất		*Mảng con thứ hai*
các phần tử nhỏ hơn x	**x**	**các phần tử lớn hơn x**

0 1 2 3 4 5 6 7 8

- Chọn phần tử **giữa** mảng, gọi là **giá trị mốc x**. Gọi vị trí **của** x là mốc.
- Phân hoạch các phần tử còn lại sao cho:
  - Các phần tử thuộc **(a[0],a[mốc-1])** đều có nội dung **nhỏ hơn x** (gọi là mảng con thứ nhất).
  - Các phần thuộc (**a[mốc+1], a[n-1]**) đều có nội dung lớn hơn hoặc bằng phần x (gọi là mảng con thứ hai).
- Quá trình lại tiếp tục như thế với mảng con thứ nhất và mảng con thứ hai.
- Kết quả sau cùng, ta sẽ được mảng đã sắp thứ tự.

GIẢI THUẬT

CÁC BƯỚC THỰC HIỆN

```
Bước 1:
   Chọn tùy ý một phần tử a[k] trong dãy là giá trị mốc
   //với điều kiện left ≤ k ≤ right
   Cho x=a[k], i=left, j=right.
Bước 2:
//Tìm và hoán vị cặp phần tử a[i] và a[j] đứng sai thứ tự.
   B2-1: Chừng nào a[i]<x thì i++; //Tìm a[i]≥x
   B2-2: Chừng nào a[j]>x thì j--; //Tìm a[j]≤x
   B2-3: Nếu i<j thì Hoanvi(a[i],a[j])//a[i],a[j] sai thứ tự
Bước 3:
   Nếu i<j: Lặp lại Bước 2;
   Nếu i≥j: thì dừng
```

MINH HỌA

- Ví dụ: cho mảng số nguyên a = {15, 10, 14, 4, 9, 1, 3, 2, 7}
- Thực hiện giải thuật Quick Sort trên mảng a

Phân hoạch đoạn [0,8]:

15	10	14	4	9	1	3	2	7
0	1	2	3	4	5	6	7	8
i=0								j=8

7	10	14	4	9	1	3	2	15
0	1	2	3	4	5	6	7	8
	i=1						j=7	

7	2	14	4	9	1	3	10	15
0	1	2	3	4	5	6	7	8
		i=2				j=6		

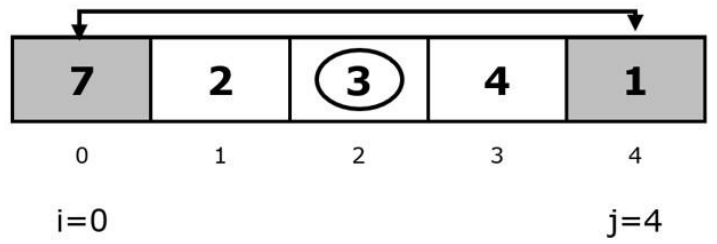
7	2	3	4	9	1	14	10	15
0	1	2	3	4	5	6	7	8
				i=4	j=5			

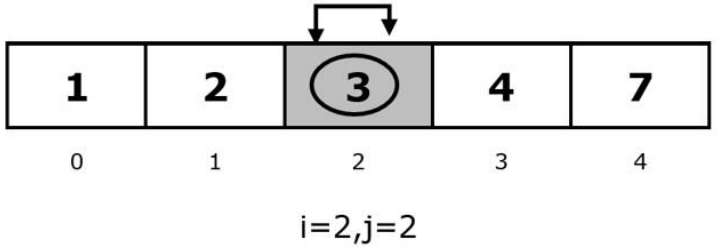
Kết quả ta được hai đoạn như sau:

đoạn [0,4]					đoạn [5,8]			
7	2	3	4	1	9	14	10	15
0	1	2	3	4	5	6	7	8
				j=4	i=5			

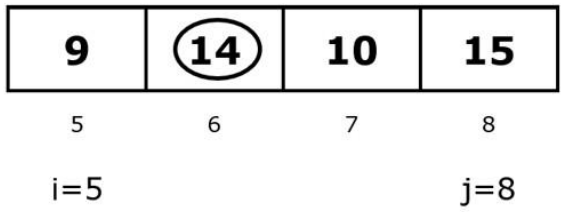
MINH HỌA

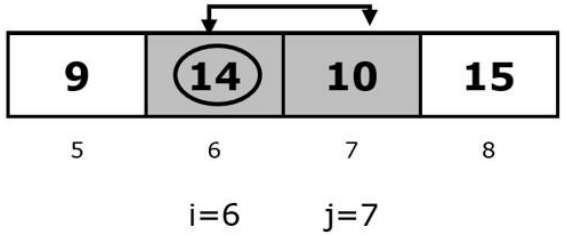
Phân hoạch đoạn [0,4]:



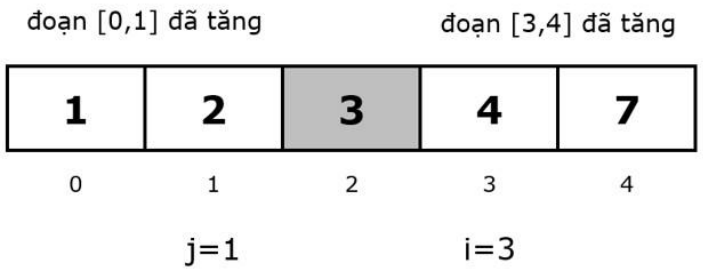


Phân hoạch đoạn [5,8]:

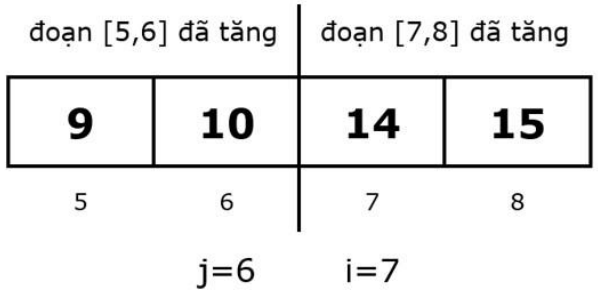




Kết quả phân hoạch đoạn [0,4]:



Kết quả phân hoạch đoạn [5,8]:



MINH HỌA

- Sau khi kết thúc phân hoạch, ta được mảng đã được sắp xếp tăng, như sau:

đoạn [0,1] đã tăng		đoạn [3,4] đã tăng			đoạn [5,6] đã tăng		đoạn [7,8] đã tăng	
1	**2**	**3**	**4**	**7**	**9**	**10**	**14**	**15**
0	1	2	3	4	5	6	7	8

CÀI ĐẶT

```
void QuickSort(int a[], int left, int right)
{
    int i, j, x;
    x=a[(left+right)/2];
    i=left, j=right;
    do{
        while(a[i]<x) i++;
        while(a[j]>x) j--;
        if(i<=j)
        {
            Hoanvi(a[i], a[j]);
            i++; j--;
        }
    } while(i<j);
    if(left<j) QuickSort(a, left, j);
    if(i<right) QuickSort(a, i, right);
}
```

NHẬN XÉT

TRƯỜNG HỢP	ĐỘ PHỨC TẠP
Tốt nhất	n*log(n)
Trung bình	n*log(n)
Xấu nhất	n^2

- Hiệu quả Quick Sort phụ thuộc việc chọn mốc x.
- Trường hợp **tốt nhất**: nếu chọn mốc là phần tử **median** (phần tử lớn hơn (hay bằng) nửa số phần tử, và nhỏ hơn (hay bằng) nửa số phần tử còn lại), khi đó dãy chia thành hai đoạn **bằng** nhau và cần **log2(n)** lần phân hoạch thì sắp xếp xong.
- Trường hợp **xấu nhất**: xảy ra khi chọn phải mốc x là **giá trị cực đại** (hoặc cực tiểu). Lúc đó dãy sẽ bị phân chia thành hai phần không đều, một phần chỉ có **một** phần tử, phần còn lại gồm **(n-1) phần tử**. Do vậy sẽ tốn **n^2 lần phân hoạch** mới sắp xếp xong.

BÀI TẬP tại cá nhân tại lớp

- Áp dụng thuật toán **QUICK SORT** cho mảng a và ghi ra kết quả từng bước

Mảng a = **{10, 5, 30, 70, 40, 80, 90}**

BÀI TẬP

- **BÀI 1**: Lần lượt áp dụng các giải thuật sắp xếp đã học, hãy chạy từng bước quá trình sắp xếp dãy số sau:

7	2	12	14	4	9	11	13	6	15	3	8
0	1	2	3	4	5	6	7	8	9	10	11

- **BÀI 2**: Trong ba phương pháp sắp xếp: chọn trực tiếp, chèn trực tiếp, **nổi bọt**, thì phương pháp nào thực hiện sắp xếp nhanh nhất với một **mảng đã có thứ tự**? Giải thích?

- **BÀI 3**: Cài đặt các thuật toán sắp xếp đã trình bày. Viết code đếm số phép so sánh và phép gán của từng thuật toán khi chạy trên cùng một mảng dữ diệu.

BÀI TẬP

BÀI 4: Viết chương trình minh hoạ các phương pháp sắp xếp, chương trình có các chức năng chính như sau:

a) Sinh mảng ngẫu nhiên gồm n số.

b) Chọn phương pháp sắp xếp, sau khi chạy xong, chương trình có báo thời gian chạy.

c) Xem danh sách sau khi đã sắp xếp.

d) Kết thúc chương trình.

BÀI 5: Viết chương trình nhập vào danh sách sinh viên, thông tin của sinh viên bao gồm: mã số sinh viên, họ tên, điểm trung bình.

a) Sắp xếp danh sách sinh viên tăng dần theo họ tên.

b) Sắp xếp danh sách sinh viên giảm dần theo điểm.

Q&A!